

Project: [ProjectName] — .NET / ASP.NET Core

Overview

[Brief description of what this project does]

Tech Stack

- **Runtime:** .NET 9
- **Framework:** ASP.NET Core 9 (Minimal APIs + Controllers)
- **ORM:** Entity Framework Core 9 (PostgreSQL via Npgsql)
- **CQRS:** MediatR 12
- **Validation:** FluentValidation
- **Testing:** xUnit + Testcontainers + FluentAssertions
- **Auth:** ASP.NET Core Identity + JWT Bearer

Project Structure

```
src/  
├── Domain/           # Entities, Value Objects, Domain Events, Interfaces  
├── Application/       # Use Cases, Commands, Queries, DTOs, Validators  
├── Infrastructure/    # EF Core, External APIs, Email, Caching  
└── API/              # Endpoints, Middleware, Configuration  
tests/  
├── Unit/             # Fast unit tests, no I/O  
└── Integration/      # Tests with Testcontainers (real DB)
```

Code Standards

Must follow:

- All async methods accept `CancellationToken ct` as last parameter
- Never use `.Result`, `.Wait()`, or `.GetAwaiter().GetResult()`
- Use `ILogger<T>` for all logging — never `Console.Write`
- `record` types for DTOs and request/response objects
- Primary constructor syntax for services and handlers (.NET 8+)
- Enable `<Nullable>enable</Nullable>` — handle all nullable warnings
- All public APIs have XML doc comments

Naming Conventions

Thing	Convention	Example
Interface	I prefix	IUserRepository
Command	CommandName + Command	CreateUserCommand
Command Handler	CommandName + Handler	CreateUserHandler
Query	QueryName + Query	GetUserByIdQuery
DTO	Descriptive suffix	UserDto , CreateUserRequest
Test method	Method- Name_Scenario_Expected	Cre- ateUser_WithDuplicateEmail_ ThrowsConflict

Architecture Rules

- Domain layer has NO dependencies on other layers
- Application layer depends only on Domain
- Infrastructure implements interfaces from Domain/Application
- API layer is the composition root — wires everything together

Common Commands

```
# Build
dotnet build

# Run tests
dotnet test

# Apply EF migrations
dotnet ef database update --project src/Infrastructure --startup-project src/API

# Add migration
dotnet ef migrations add [MigrationName] --project src/Infrastructure --startup-project src/API

# Run locally
dotnet run --project src/API
```

Configuration

- Connection strings in appsettings.Development.json (never committed)
- Secrets via dotnet user-secrets or environment variables
- Production config via environment variables only

Do NOT

- Hardcode connection strings or API keys
- Put business logic in controllers/endpoints
- Use `static` mutable state
- Catch `Exception` (catch specific exceptions)
- Suppress nullable warnings with `!` without comment explaining why